

Setup 2 — Zephyr Workspace and Folder Structure

Goal

Understand what `west init` and `west update` create on disk, what every top-level folder in the workspace contains, and which parts of `myworkspace/zephyr/` you will read and modify when writing firmware.

What you will learn

- What a west workspace is and how it differs from a single git repo
- The role of every folder west creates in `myworkspace/`
- Which subdirectories of `myworkspace/zephyr/` matter for daily development
- Where board definitions, drivers, samples, and Kconfig files live

Overview

Running `west init` followed by `west update` populates your workspace with the Zephyr kernel and all its dependencies. The result is a multi-repo workspace managed by west — similar to how Android uses repo or ROS uses catkin_ws.

Workspace top level

<code>myworkspace/</code>	
├─ <code>.venv/</code>	← Python virtual environment (west,
<code>pyelftools</code> , etc.)	
├─ <code>.west/</code>	← west configuration (do not edit
manually)	
│ └─ <code>config</code>	← records the manifest repo URL and local
<code>paths</code>	
├─ <code>zephyr/</code>	← Zephyr RTOS source (kernel, drivers,
<code>boards</code> , <code>samples</code>)	
└─ <code>modules/</code>	← External dependencies fetched by west

		└─ hal/	← Hardware Abstraction Layers (HAL)
		└─┬─ nordic/	← nRF MDK (Nordic-specific register
		definitions)	
		└─┬─ cmsis/	← ARM CMSIS headers (Cortex-M core access)
		└─ fs/	← File system modules (LittleFS, FatFS)
		└─ lib/	← Libraries (zcbor CBOR encoder, OpenAMP,
		etc.)	
		└─ crypto/	← mbedTLS, TinyCrypt
		└─ tee/	← Trusted Execution Environment support
		└─ bootloader/	
		└─┬─ mcuboot/	← MCUBoot secure bootloader (OTA update
		support)	
		└─ tools/	← Build-time host tools (CMake modules,
		Jinja, etc.)	

Each subdirectory of `modules/` and `bootloader/` is its own git repository, pinned to a specific commit recorded in `zephyr/west.yml`. Running `west update` checks out those exact commits — you always get a reproducible, tested combination.

Inside `myworkspace/zephyr/`

This is the Zephyr OS itself. You read it often; you rarely edit it.

zephyr/	
└─ arch/	← CPU architecture ports (ARM, RISC-V,
x86, ARC, ...)	
└─ arm/	← Cortex-M startup code, exception
handlers, MPU	
└─ boards/	← Board definitions (DTS + Kconfig +
defconfig)	
└─ nordic/	
└─┬─ nrf52840dk/	← The nRF52840 DK you use in this course
└─┬─ nrf52840dk_nrf52840.dts	← hardware
description	
└─ nrf52840dk_nrf52840_defconfig	← default Kconfig
options	
└─ board.cmake	← how to flash this
board	
└─ drivers/	← Peripheral drivers (GPIO, I2C, SPI,
UART, BLE, ...)	
└─ gpio/	
└─ i2c/	
└─ sensor/	← Sensor framework drivers (BME280 lives
here)	
└─ bluetooth/	
└─ dts/	← Device Tree includes and bindings
└─ arm/nordic/	← nRF52840 SoC DTS (peripheral base
addresses, IRQs)	
└─ bindings/	← YAML files describing each compatible=

```

string
├─ include/           ← Public API headers your application
#include
├─   └─ zephyr/
├─       └─ kernel.h   ← k_thread_create, k_sem_take,
k_sleep, ...
├─       └─ drivers/   ← gpio.h, i2c.h, spi.h, sensor.h, ...
├─           └─ bluetooth/ ← bt_enable, bt_le_adv_start, gatt.h, ...
├─ kernel/           ← Scheduler, threads, IPC, timers (the
RTOS core)
├─ subsys/           ← Higher-level subsystems
├─   └─ bluetooth/   ← BLE host stack (HCI, L2CAP, GATT, GAP)
├─   └─ usb/         ← USB device stack (CDC-ACM, HID, MSC)
├─   └─ shell/       ← Interactive shell subsystem
├─   └─ fs/          ← NVS, LittleFS, FatFS integration
├─       └─ logging/  ← LOG_INF / LOG_ERR backend
├─ lib/              ← Portable libraries (CBOR/zcbor, ring
buffers, etc.)
├─ samples/          ← Working reference examples for every
subsystem
├─   └─ basic/blinky/ ← The hello-world of Zephyr
├─   └─ bluetooth/   ← BLE peripheral, central, beacon examples
├─       └─ drivers/  ← GPIO, I2C, SPI, ADC sample apps
├─ scripts/          ← Python tools (west extensions, DTS
scripts, runners)
├─ soc/              ← SoC-level definitions (nRF52840 clock,
power, pinctrl)
├─ cmake/            ← CMake modules (FindZephyr-sdk, toolchain
selection)
├─ Kconfig            ← Root Kconfig file (entry point for
menuconfig)
├─ CMakeLists.txt     ← Root CMake (included by every app's
build)
├─ west.yml           ← Declares all module dependencies and
their versions

```

The folders you will use most

Folder	When you use it
boards/nordic/ nrf52840dk/	Reading the board DTS to understand pin mappings
drivers/sensor/ bme280/	Reading driver source to understand the sensor API
include/zephyr/	Following #include paths to find function signatures
dts/bindings/	Reading YAML to understand which compatible= strings exist

Folder	When you use it
<code>samples/</code>	Copying a working example as a starting point
<code>subsys/bluetooth/</code>	Tracing BLE stack behaviour when debugging

Your application vs Zephyr source

Your application code (in `src/`) sits **outside** the `myworkspace/zephyr/` tree. CMake links them together at build time:

```

myworkspace/
├─ zephyr/           ← Zephyr source (read-only, managed by
west)                west)
├─ (your project)/
│   └─ src/
│       ├── main.c    ← your code
│       └─ prj.conf   ← selects Kconfig options from zephyr/
Kconfig
├─ CMakeLists.txt ← find_package(Zephyr) pulls in the OS
build
└─ app.overlay ← extends/overrides the board DTS

```

`find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})` in your `CMakeLists.txt` is the line that connects your application to the Zephyr build system and SDK.

`west.yml` pins every dependency

```

# zephyr/west.yml (excerpt)
manifest:
  projects:
    - name: hal_nordic
      revision: abc123def ← exact commit, reproducible build
    - name: mcuboot
      revision: v2.1.0
    - name: zcbor
      revision: 0.9.0

```

Running `west update` checks out these exact revisions. This is why you can share a project with a colleague and get identical builds even months later.

Why this matters

Knowing where to look speeds up debugging dramatically. When `LOG_ERR` says “sensor not ready”, you can open `drivers/sensor/bme280/bme280.c` directly to read the init code. When a DTS binding is unclear, you open `dt/bindings/sensor/bosch,bme280.yaml` to see every valid property. Reading Zephyr source is a core skill — it is well-written and well-commented.

Practice tasks

1. Open `myworkspace/zephyr/boards/nordic/nrf52840dk/nrf52840dk_nrf52840.dts` and find which GPIO pin `led0` is mapped to.
2. Open `myworkspace/zephyr/include/zephyr/drivers/gpio.h` and read the signature of `gpio_pin_configure_dt` — match it to how you use it in Day 9.
3. Browse `myworkspace/zephyr/samples/bluetooth/peripheral/` and compare its `prj.conf` to your Day 18 `prj.conf`.
4. Open `myworkspace/zephyr/west.yml` and find the pinned revision for `hal_nordic` — this is the nRF MDK version your build uses.